Universidad de Granada
Departamento de Ciencias de la Computación
e Inteligencia Artificial

DECSAI

Curso 2024/25

Técnicas de los Sistemas Inteligentes

Enunciado de la Práctica 2

En esta práctica, el alumnado debe resolver seis problemas usando MiniZinc. Cada ejercicio se resolverá en un **fichero MZN por separado (siguiendo la nomenclatura indicada)**. La solución entregada debe ser **correcta y completa**, es decir, todas las soluciones válidas son admitidas por la codificación entregada y ésta no admite ninguna solución no válida. **El código debe estar perfectamente comentado, explicando y justificando la solución presentada, variables y restricciones usadas para codificar cada problema, y cualquier otro aspecto relativo a las decisiones tomadas durante el proceso de codificación. Un código cuyos comentarios sean, a criterio del profesor, deficientes podrá invalidar total o parcialmente aquellos problemas a los que afecte.** Para la resolución de todos los problemas se recomienda usar el **solver Gecode**, solver pre-instalado en el IDE de Minizinc, con sus valores por defecto.

Nota1: Los problemas con **errores sintácticos** en MiniZinc automáticamente califican con una puntuación de **0 (cero) puntos**.

Nota2: Como ya se indicó en la Práctica 1, no supone ningún problema utilizar ChatGPT u otras herramientas de IA generativa como apoyo para el desarrollo de las prácticas. De hecho, incluso se recomienda su uso, dado que se trata, sin duda, de herramientas de gran utilidad en la realización de múltiples tareas, y que nos permiten aumentar nuestra productividad en gran medida. No obstante, cuando se empleen, se debe indicar con claridad y especificar de qué modo y en qué partes se han usado (y por qué). Lo contrario, puede suponer una penalización severa en la calificación recibida (incluyendo el suspenso de la práctica y/o de la asignatura). Véase https://ceprud.ugr.es/formacion-tic/inteligencia-artificial (especialmente, la sección titulada "Recomendaciones e ideas para estudiantes").

Junto con lo anterior, se entregará una **memoria en formato PDF** (con un **tamaño máximo de 10 páginas**[1]) donde se describa las respuestas a las preguntas planteadas en cada ejercicio de forma razonada (no se incluirá código; éste ya se entrega en el fichero MZN). En caso de que no se plantee ninguna pregunta conceptual concreta en un ejercicio, en la memoria se deben indicar los resultados obtenidos así como una explicación de la codificación empleada.

Cada problema tiene una **puntuación que se indica en el propio título del ejercicio**, y dicha puntuación incluye tanto la codificación MZN como la respuesta razonada en la memoria. La entrega consistirá en un **fichero ZIP** que contenga la memoria de la práctica (PDF) y los ficheros MZN de cada problema.

Fecha de entrega: 11/05/2025 23:59

Modo de entrega: A través de la plataforma **PRADO**, en la tarea creada para ello.

[1] Estas 10 páginas incluyen todo. Es decir, si se quiere incluir una portada y un índice, o una sección bibliográfica, esas páginas se contabilizarían dentro del máximo de 10.

1. Problema del cambio de monedas (2 ptos)

a. Se desea encontrar un conjunto de monedas cuyo importe sea exactamente una cantidad dada. Para ello, partimos de una divisa imaginaria (que llamaremos Unidad), en donde se dispone de monedas de 1, 3, 5, 10, 25 y 50 céntimos, así como de las de 1, 2 y 5 Unidades (asumiendo que 1 Unidad equivale a 100 céntimos, 2 Unidades a 200 céntimos, y 5 Unidades a 500 céntimos). Codifique una solución en MZN (**empleando, para cada importe y cada moneda, codificación entera**) que calcule cualquier solución válida, y complete la siguiente tabla:

Importe	Primera solución encontrada y número de monedas en la misma	Número total de soluciones	Runtime (en milisegundos)
0,13 Unid.			
1,47 Unid.			
2,30 Unid.			
2,99 Unid.			

Nota1: Para calcular todas las soluciones (en lugar de una sola), se debe desmarcar la casilla "*Stop after this many solutions (uncheck for all)*" en el menú "*Show configurator editor*" (*"User defined behavior"*). Igualmente, para imprimir el número total de soluciones, debe activarse la opción "*Output solving statistics*", también en este menú. El número de soluciones aparece en la consola en el campo "%%%mzn-stat: nSolutions" (cuando se ejecuta con el solver Gecode).

Nota2: Para el *runtime*, se recomienda usar el valor reportado por MiniZinc en la última línea "Finished in …" (ya que el valor reportado en las estadísticas como "solveTime" no incluye todo el proceso de resolución y, a veces, es algo más inestable por la forma en que es medido).

b. Modifique el código del apartado anterior de forma que la parte entera de los importes sea asignada únicamente a monedas de una o dos Unidades (dado que la de 5 Unidades, para los importes solicitados, no es necesaria). Por ejemplo, para el importe de 1,47 Unidades se deberá forzosamente asignar una moneda de 1 Unidad y, posteriormente, completar los 47 céntimos restantes con monedas de 1, 3, 5, 10, 25 o 50 céntimos. Complete la tabla anterior con esta nueva codificación.

c. Realice una nueva modificación sobre la versión anterior (la del apartado (b)) para que el conjunto de monedas encontrados sea mínimo, es decir, que tenga el menor número de monedas posible. Para ello, se debe minimizar la variable que guarda la "cantidad" de monedas de la solución encontrada empleando:

solve minimize <variable>

A continuación, se debe generar una nueva tabla de resultados con una columna de "solución óptima", donde se indique la solución óptima encontrada para los importes anteriores y la cantidad de monedas de la misma, y otra columna de *runtime*.

Importe	Solución óptima	Número de monedas de la solución óptima	Runtime
0,13 Unid.			
1,47 Unid.			
2,30 Unid.			
2,99 Unid.			

d. Responda razonadamente a las siguientes cuestiones e implemente el código MiniZinc que se solicita en el punto (iii):
   i. A medida que vamos aumentando el importe en el apartado (a), ¿qué ocurre? ¿Cómo escala (a nivel de tiempos de resolución) nuestra codificación con importes progresivamente más elevados? Por ejemplo, ¿cuánto se tardaría en encontrar **todas** las soluciones para un importe de 3.5 Unid. y 5.27 Unid.?
   ii. En relación con el punto anterior, ¿qué ocurriría si, usando la codificación (a) para encontrar **todas** las soluciones, el importe buscado es mucho mayor (del orden de millones de Unidades)?
   iii. En caso de que haya algún problema con lo planteado en el punto anterior (es decir, si resulta problemático encontrar todas las soluciones para un importe de millones de Unidades), ¿cuál podría ser una estrategia prometedora para resolverlo? Nos referimos a alguna estrategia que permita reducir el espacio de búsqueda y obtener soluciones en un tiempo razonable (aún a costa de no obtener todas las posibles soluciones con todas las monedas). Por ejemplo, el alumnado puede tomar como referencia el apartado (b) de este ejercicio para responder a la pregunta y generar un código MiniZinc que, efectivamente, permita resolver el problema para un importe de, por ejemplo, 122233367 Unidades en menos de 1 segundo.

e. Finalmente, el alumnado debe repetir la implementación del apartado (a) pero, en esta ocasión, con **codificación flotante** (tanto para el valor de las monedas, como para los importes y el número de monedas usadas). ¿Qué ocurre ahora y a qué se puede deber?

Se deben entregar todos los ficheros MZN con la siguiente nomenclatura Ejercicio1a.mzn, Ejercicio1b.mzn, Ejercicio1c.mzn, Ejercicio1d.mzn y Ejercicio1e.mzn. La salida debe formatearse de modo que, al ejecutar el código, se puedan ver los resultados del siguiente modo:

monedas: "[nº monedas de 1 céntimo, nº monedas de 3 céntimos, nº monedas de 5 céntimos, nº monedas de 10 céntimos, nº monedas de 25 céntimos, número monedas de 50 céntimos, nº monedas de 1 Unidad, nº monedas de 2 Unidades, nº monedas de 5 Unidades]"

total monedas: "número total de monedas de la solución"

2. Puzzle lógico (1.5 ptos)

Supongamos que el 26 de Agosto, se reúnen en Granada cinco parejas para celebrar sus aniversarios de boda. Cada pareja viene de distintos sitios: Madrid, Barcelona, Sevilla, Lyon, y Parma. El nombre de uno de los miembros de la pareja es Lucía, Sofía, Juan, Cristina y Claudia. Los otros miembros de cada pareja serían Pablo, Marcos, José, Marta y Pedro. Debemos tener en cuenta que no hay dos parejas que lleven casadas la misma cantidad de años. A partir de las pistas siguientes, debemos determinar quiénes son las personas (nombres) que conforman cada pareja, la ciudad de la que vienen, y cuántos años llevan casados:

1) Sofía no lleva casada tanto tiempo como José o los de Barcelona, pero sí más que Marcos o los Madrid.
2) Cristina lleva casada el doble de tiempo que los de Sevilla, pero solo la mitad que Pablo.
3) Los de Parma llevan casados diez años más que Marta y diez años menos que Juan.
4) Marcos y Claudia llevan 25 años menos de casados que los de Lyon, quienes, con 30 años de casados, son la pareja que lleva más tiempo casada.
5) Ni Lucía ni los de Madrid son los que llevan casados menos tiempo.
6) Pedro lleva 25 años casado.

La salida de MiniZinc se debe formatear de modo que se muestre, para cada pareja, la siguiente información:

"Ciudad": "Nombre1" y "Nombre2" "Número" años

El fichero entregado se debe denominar Ejercicio2.mzn.

3. Problema de la tabla binaria (1.5 ptos)

Disponemos de una matriz 12×12 con valores binarios. Dicha tabla debe ser rellenada con ceros y unos cumpliendo los siguientes requisitos:

1) Se parte de una serie de valores preasignados que hay que respetar. En este caso:

	1	0					0			0	
	1	1			1						
								1			0
		0	0								0
						1	1		0		
	1		0		1	1				1	
								1			
1			1							0	
	1							0			
							0				
1											1
	1		1						0	0	

Esta sería la plantilla de valores que se nos proporciona como punto de partida.

2) Cada fila y cada columna debe contener el mismo número de ceros que de unos. Cada columna tiene *k* unos/ceros y cada fila tiene también *k* unos/ceros.

3) Ninguna fila puede ser idéntica a ninguna otra fila, ni ninguna columna puede ser idéntica a ninguna otra columna.
4) Ninguna fila contiene 3 o más valores consecutivos iguales. Lo mismo ocurre con las columnas. Ninguna columna contiene 3 o más valores consecutivos iguales.

¿Cuántas soluciones tiene este problema? Indique en la memoria el número de posibles soluciones válidas, así como las matrices 12×12 resultantes. El fichero entregado se debe denominar Ejercicio3.mzn, y debe proporcionar como salida las matrices binarias resultantes.

4. Problema de los cuadrados (1 pto)

Podemos definir un cuadrado perfecto x^2 como uno que satisfaga $x^2 = \sum\{y^2 \mid y \in S\}$, siendo S un conjunto finito de números naturales (distintos) más pequeños que x. Por ejemplo, si $x = 5$, entonces S sería $\{4, 3\}$. Es decir, $5^2 = 4^2 + 3^2$. Se busca obtener aquellas soluciones que se correspondan con el conjunto S más grande (de mayor cardinalidad). Por ejemplo, si $x = 10$, la solución buscada sería $\{7, 5, 4, 3, 1\}$ y no $\{8,6\}$, a pesar de que ambos proporcionarían $100 = 10^2 = 7^2 + 5^2 + 4^2 + 3^2 + 1^2 = 8^2 + 6^2$.

Se debe implementar dicho problema de optimización de restricciones y completar la siguiente tabla:

X	S	|S|	Runtime
4			
5			
6			
7			
8			
9			
10			
15			
30			
60			
120			

En caso de que el tiempo (*runtime*) pase de 5 minutos, puede detenerse la ejecución e indicar en la tabla "-" en S y |S|, e ∞ en *runtime*. En la memoria, aparte de la tabla cubierta, se debe crear una gráfica que muestre en el eje de abscisas los valores de X y en el eje de ordenadas el *runtime*. ¿Qué conclusión se puede extraer del comportamiento observado? ¿Se trata de un problema escalable? Intente reflexionar sobre la complejidad computacional del problema en cuestión, y relaciónelo con los resultados obtenidos.

La ejecución del código debe proporcionar una salida formateada del siguiente modo:

```
x = "numero propuesto"
S = {["dígitos del conjunto S separados por comas"]}
Cardinalidad = "número de elementos en el conjunto S"
Suma de cuadrados = "resultado de la suma de cuadrados"
```

El fichero entregado se debe denominar Ejercicio4.mzn.

5. Problema de planificación de tareas (2 ptos)

a. El legendario DeLorean de "Regreso al Futuro"[2] debe ser ensamblado lo más rápido posible para poder viajar al pasado. Para ello, contamos con tres trabajadores: Marty McFly, Doc Brown y Biff Tannen (en calidad de trabajador de apoyo, cuyo rol se detallará más adelante). La siguiente tabla muestra las tareas necesarias para construir el DeLorean, junto con la duración de cada una dependiendo de quién la realice, y las tareas que deben completarse antes de que otra pueda comenzar. Cada tarea solo puede ser realizada por un trabajador (que, además, no puede hacer dos tareas a la vez), pero los dos (Marty y Doc) pueden trabajar simultáneamente en diferentes tareas.

Queremos encontrar una asignación óptima de tareas y tiempos de inicio para completar la construcción en el menor tiempo posible.

Tarea	**Duración en días (Marty, Doc)**	**Predecesoras**
A – Montar chasis	10, 16	Ninguna
B – Instalar ruedas	1, 2	A
C – Cableado eléctrico	4, 3	B
D – Motor de fusión nuclear	15, 5	A,C
E – Tablero de control	14, 3	C
F – Instalación y configuración del condensador de fluzo (*flux capacitor*)	13, 5	C, D, E
G – Ajuste aerodinámico	7, 6	C,D
H – Instalación de puertas de ala de gaviota	3, 4	G
I – Panel de tiempo	2, 2	F,H
J – Implementación y validación de algoritmos de búsqueda heurística para optimización de caminos	4, 2	G, I

Biff podrá ser asignado para apoyar a Doc o a Marty en cualquier tarea con duración mayor o igual a 3 días (es decir, si la duración de la tarea para el trabajador asignado es menor que 3 días, Biff no podrá ser asignado a la misma). Gracias a la ayuda de Biff, se reducirá la duración de la tarea en cuestión en dos días (para las tareas A, C y H) o un día (para todas las demás). Por ejemplo, si la tarea de montar el chasis es asignada a Marty, y Biff le ayuda, la duración de esta tarea sería de 8 días (en lugar de 10 días). Biff solo puede apoyar en un máximo de 4 tareas en total y nunca en tareas que se realicen simultáneamente.

En la memoria, se debe indicar con claridad cuál es la duración mínima de la construcción del DeLorean, y se debe incluir un diagrama de Gantt con la planificación encontrada (véase https://en.wikipedia.org/wiki/Gantt_chart). Para este apartado, se debe entregar un fichero denominado Ejercicio5a.mzn, que imprima la siguiente información (en caso de que se lleve a cabo con la ayuda de Biff, y sin esa última parte ("con apoyo de Biff") en caso contrario):

```
"NombreTarea" inicio día "X" fin día "Y". Asignada a "NombreTrabajador" con apoyo de Biff
```

Por ejemplo, podríamos tener `"Montar chasis" inicio día "1" fin día "8". Asignada a "Marty" con apoyo de Biff`, lo que significaría que la tarea "Montar chasis" comienza el día 1

[2] https://en.wikipedia.org/wiki/Back_to_the_Future

y termina el 8 (incluido), y la llevan a cabo Marty y Biff. Esta tarea a Marty le llevaría 10 días pero, como le ayuda Biff, le lleva dos días menos.

b. Una vez haya resuelto este problema de optimización de restricciones (COP), plantéelo como un problema de satisfacción de restricciones (CSP) siguiendo las indicaciones de la presentación de la práctica, y calcule el número de soluciones existentes con ese valor mínimo. Es decir, ¿cuántas configuraciones/planes diferentes existen con ese coste mínimo? Se debe entregar un fichero denominado Ejercicio5b.mzn, que resuelva el CSP e imprima todos los planes de tiempo mínimo.

6. Problema de conformación de tribunales de TFG (2 ptos)

Se acerca Julio y, por tanto, el momento en que muchos estudiantes defenderán sus Trabajos de Fin de Grado (TFGs). Para ello es necesario configurar los tribunales de evaluación, teniendo en cuenta que hay un total de 10 TFGs a evaluar en esta convocatoria. De acuerdo a la normativa, cada tribunal puede evaluar un máximo de 5 TFGs, y cada tribunal evaluador consta de 3 personas, cada una de un departamento diferente (DECSAI, LSI e ICAR). Se ha decidido que habrá 2 tribunales de evaluación, y disponemos de 9 profesores (3 de cada departamento: profesores 1-3 de DECSAI, profesores 4-6 de LSI, y profesores 7-9 de ICAR). Las defensas de TFG tendrán lugar en una semana concreta (determinada por el calendario de evaluación) y pueden ser cualquier día de la semana, de Lunes a Viernes, siempre y cuando a los propios profesores les venga bien esa fecha. Todo esto que, a priori, suena muy sencillo, en realidad es mucho más complicado de lo que parece:

- Primero, porque cada profesor tiene una apretada agenda (con obligaciones docentes, investigadoras y de gestión) que le impide estar disponible todos los días. En este sentido,
    - El profesor 1 solo podría participar en el tribunal el martes.
    - Los profesores 2 y 3 solo pueden lunes, miércoles y viernes.
    - El profesor 4 solo miércoles y jueves.
    - El profesor 5 solo lunes y jueves.
    - El profesor 6 es el único que está disponible todos los días.
    - El profesor 7 solo el viernes.
    - El profesor 8 solo los miércoles y jueves.
    - El profesor 9 solo está disponible los jueves.

- En segundo lugar, existe la regla de que un profesor que haya supervisado/dirigido un TFG no puede formar parte del tribunal que lo evalúe. En este caso:
    - El profesor 2 ha dirigido los TFGs 1 y 10.
    - El profesor 3 ha dirigido el TFG 8 y ha codirigido el TFG 7 con el profesor 8.
    - El profesor 4 ha dirigido el TFG 6 y ha codirigido el TFG 4 con el profesor 8.
    - El profesor 5 ha dirigido el TFG 2.
    - El profesor 6 ha codirigido con el profesor 7 el TFG 3.

    Esto significa que, por ejemplo, el profesor 2 no puede formar parte del tribunal que evalúe el TFG 10.

- Y, junto con lo anterior, existen otra serie de restricciones, acordadas por la Escuela y los departamentos, con el objetivo de que la evaluación de TFGs sea lo más justa y llevadera para todas las partes:
    - Un profesor solo puede estar en un único tribunal evaluador.

- El día de evaluación debe ser diferente para cada tribunal. Es decir, el mismo día no puede haber dos tribunales ejerciendo.

Responda en la memoria a las siguientes preguntas de forma razonada:

- ¿Cuál es el número de soluciones válidas obtenidas? En caso de ser menos de 10, lístelas explícitamente.
- Discuta la posible existencia de soluciones simétricas. Por soluciones simétricas se entienden aquellas que tienen valores distintos para las variables de la codificación CSP (por lo que MiniZinc las interpreta como soluciones diferentes), pero semánticamente representan la misma solución[3]. En caso de que la codificación propuesta por el estudiante contenga soluciones simétricas, ¿cómo se podrían evitar y cuál es el número de soluciones (no simétricas) obtenido? Explique cómo se consigue la rotura de simetrías[4] (variables y/o restricciones utilizadas para ello), y entregue la solución MZN sin simetrías. En caso de que la solución inicial directamente no contuviese simetrías, analice de qué modo sí podría haberlas (es decir, qué podría provocar la existencia de simetrías en este problema) y de qué forma la codificación escogida implícitamente las elimina.

Para este apartado, se debe entregar un fichero denominado Ejercicio6.mzn, que imprima la siguiente información:

```
Tribunal 1: "numero profesor"("depto"), "numero profesor"("depto"),"numero
profesor"("depto") (Día: "dia de la semana")
TFGs a evaluar: "numeros de TFG separados por comas"

Tribunal 2: "numero profesor"("depto"), "numero profesor"("depto"),"numero
profesor"("depto") (Día: "dia de la semana")
TFGs a evaluar: "numeros de TFG separados por comas"
```

[3] Por ejemplo, si tuviésemos Solución1 (Tribunal 1: {Prof1, Prof2, Prof3} a evaluar TFGs 1-5, Tribunal 2: {Prof4, Prof5, Prof6} a evaluar TFGs 6-10) y Solución2 (Tribunal 1: {Prof3, Prof2, Prof1} a evaluar TFGs 1-5, Tribunal 2: {Prof4, Prof5, Prof6} a evaluar TFGs 6-10). En realidad, representarían la misma solución, y solo supondrían una permutación del orden de los profesores del tribunal 1. En este caso, MiniZinc estaría considerando como soluciones diferentes dos soluciones semánticamente idénticas. Esto suele ser resultado de una codificación inadecuada por nuestra parte. Si tenemos esto en cuenta, podremos entender por qué es interesante romper simetrías: porque nos transmiten la impresión de que hay más soluciones de las que realmente hay.

[4] https://docs.minizinc.dev/en/stable/efficient.html#symmetry